手写小GPT学习笔记

一、Scaled Dot-Product Attention 原理

Scaled Dot-Product Attention 是 Transformer 架构中注意力机制的核心组件，是一种信息聚合机制，核心逻辑是 "按相关性检索"：通过计算查询（Query）与键（Key）的相似度确定 "关注哪里"，再根据这个权重从值（Value）中提取信息 "关注什么"。

1. 核心概念

- **Query (Q)**：查询向量，代表当前需要获取信息的主体，形状通常为 `[batch_size, seq_len_q, d_k]`
- **Key (K)**：键向量，代表可供查询的信息的标识，形状通常为 `[batch_size, seq_len_k, d_k]`
- **Value (V)**：值向量，代表与键对应的实际信息内容，形状通常为 `[batch_size, seq_len_v, d_v]`，在标准实现中 `seq_len_k = seq_len_v`，`d_k = d_v`

2. 计算步骤

1. **相似度计算**：计算 Q 和 K 的点积，得到原始注意力分数矩阵，衡量查询与键的匹配程度
   $scores = Q \times K^T$
   得到的分数矩阵形状为 `[batch_size, seq_len_q, seq_len_k]`
2. **缩放操作**：将分数矩阵除以 $\sqrt{d_k}$ （d_k 是 Q/K 的维度）
   $$scaled_scores = \frac{Q \times K^T}{\sqrt{d_k}}$$
   缩放的原因：当 d_k 较大时，点积结果的绝对值会变大，导致 softmax 函数进入饱和区，梯度消失。缩放操作使得点积结果的方差保持在 1 附近，稳定训练过程。
3. **Softmax 归一化**：在最后一个维度（键序列维度）上应用 softmax，将分数转换为概率分布
   $attn_weights = softmax(scaled_scores)$
   得到的注意力权重形状为 `[batch_size, seq_len_q, seq_len_k]`，所有权重之和为 1
4. **加权求和**：将注意力权重与 V 相乘，得到最终的注意力输出
   $output = attn_weights \times V$
   输出形状为 `[batch_size, seq_len_q, d_v]`

3. 掩码机制（可选）

如果需要使用掩码（比如因果掩码，防止模型看到未来的 token），会在 softmax 之前将掩码位置的分数设置为负无穷，这样 softmax 后这些位置的权重会趋近于 0。

二、手写 Scaled Dot-Product Attention 实现

1. 手动实现版本

Code block

```python
import torch
import torch.nn as nn
import math

class ScaledDotProductAttention(nn.Module):
    def __init__(self, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

    def forward(self, q, k, v, mask=None):
        # 获取键的维度
        d_k = q.size(-1)

        # 1. 计算注意力分数：QK^T / √d_k
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)

        # 2. 应用掩码（如果提供）
        if mask is not None:
            # 将掩码位置的分数设置为负无穷，softmax后这些位置权重为0
            scores = scores.masked_fill(mask == 0, -1e9)

        # 3. 应用softmax和dropout
        attn_weights = self.dropout(torch.softmax(scores, dim=-1))

        # 4. 加权求和得到输出
        output = torch.matmul(attn_weights, v)

        return output, attn_weights
```

2. 使用 PyTorch 内置函数版本

PyTorch 2.0 及以上版本提供了内置的 `torch.nn.functional.scaled_dot_product_attention` 函数，该函数有融合实现，性能更好：

Code block

```python
import torch.nn.functional as F

def scaled_dot_product_attention_builtin(q, k, v, mask=None, dropout=0.0):
```

```
    output = F.scaled_dot_product_attention(
        q, k, v,
        attn_mask=mask,
        dropout_p=dropout,
        is_causal=False  # 如果是因果掩码，设置为True
    )
    return output
```

三、测试代码

Code block

```
# 测试手动实现的Scaled Dot-Product Attention
if __name__ == "__main__":
    # 模拟输入：batch_size=2, seq_len=5, d_k=d_v=64
    batch_size = 2
    seq_len = 5
    d_model = 64

    # 随机生成Q、K、V
    q = torch.randn(batch_size, seq_len, d_model)
    k = torch.randn(batch_size, seq_len, d_model)
    v = torch.randn(batch_size, seq_len, d_model)

    # 初始化注意力模块
    attn = ScaledDotProductAttention(dropout=0.1)

    # 前向传播
    output, attn_weights = attn(q, k, v)

    print(f"输入Q形状: {q.shape}")
    print(f"输入K形状: {k.shape}")
    print(f"输入V形状: {v.shape}")
    print(f"输出形状: {output.shape}")
    print(f"注意力权重形状: {attn_weights.shape}")
    print(f"注意力权重每行和为1: {torch.allclose(attn_weights.sum(dim=-1), torch.ones_like(attn_weights.sum(dim=-1)))}")

    # 测试内置函数
    output_builtin = scaled_dot_product_attention_builtin(q, k, v, dropout=0.1)
    print(f"内置函数输出形状: {output_builtin.shape}")
```

四、代码说明

1. **手动实现**：完整实现了 Scaled Dot-Product Attention 的所有步骤，包括掩码处理和 dropout

2. **内置函数**：PyTorch 的内置函数性能更好，支持多种优化实现（如 FlashAttention）
3. **测试代码**：验证了注意力机制的输入输出形状正确性，以及注意力权重的归一化特性

五、关键注意事项

1. **维度匹配**：Q 的最后一维必须和 K 的最后一维相等，K 的倒数第二维必须和 V 的倒数第二维相等
2. **掩码形状**：掩码的形状需要能广播到注意力分数矩阵的形状 `[batch_size, seq_len_q, seq_len_k]`
3. **缩放的重要性**：不要省略缩放步骤，否则在 d_k 较大时会导致训练困难
4. **dropout**：在训练时使用 dropout 防止过拟合，推理时 dropout 设置为 0

（注：文档部分内容可能由 AI 生成）